

Quantization Techniques for Large Language Models

Rani Ramez

September 27, 2025

Introduction

Transformer-based models such as BERT and LLaMA have achieved state-of-the-art performance across a wide range of natural language processing (NLP) tasks. However, these models are extremely large, with hundreds of millions to billions of parameters, leading to significant storage and inference costs. Reducing the memory footprint and speeding up inference are therefore critical for deploying such models in real-world applications, especially on resource-constrained devices.

One of the most effective solutions is **quantization**, a model compression technique that reduces the precision of weights and activations, while preserving model accuracy.

What is Quantization?

Quantization maps continuous-valued floating-point parameters into a smaller discrete set of integer values. In standard settings, weights are stored as 32-bit floating-point (FP32). Quantization reduces this to lower precision such as 16-bit floating point (FP16), 8-bit integers (INT8), or even 4-bit integers (INT4).

Mathematically, uniform affine quantization can be described as:

$$q = \text{clip}\left(\left\lfloor \frac{x}{s} \right\rfloor + z, q_{\min}, q_{\max}\right), \quad (1)$$

$$\hat{x} = s \cdot (q - z), \quad (2)$$

where:

- x is the original floating-point value,
- s is the scale factor,
- z is the zero-point,
- $q_{\min}, q_{\max}$ are the minimum and maximum integer values,
- $\hat{x}$ is the reconstructed value after dequantization.

This process reduces storage requirements proportionally to the bit-width:

$$\text{Memory Size} \approx N \times \frac{\text{bits}}{8},$$

where N is the number of parameters.

Types of Quantization

1. Post-Training Quantization (PTQ)

PTQ applies quantization to a pre-trained model without retraining. It can be:

- **Dynamic Quantization:** Weights are stored in lower precision (e.g., INT8), while activations are quantized dynamically during inference. This requires no calibration data.
- **Static Quantization:** Both weights and activations are quantized ahead of time using a calibration dataset to determine scale and zero-point values.

2. Quantization-Aware Training (QAT)

In QAT, quantization effects are simulated during training by inserting “fake quantization” nodes. This allows the model to adapt to reduced precision and usually yields higher accuracy compared to PTQ, especially at very low bit-widths (e.g., 4-bit).

3. Weight-Only Quantization for LLMs

For very large models such as LLaMA, a common approach is weight-only quantization into 8-bit or 4-bit formats using libraries like `bitsandbytes`. Activations remain in FP16 for stability, while weights are stored in low precision to reduce GPU memory usage.

Advantages of Quantization

- **Reduced Memory Footprint:** INT8 reduces storage by $\sim 4\times$ compared to FP32, while INT4 achieves $\sim 8\times$.
- **Faster Inference:** Lower precision enables faster matrix multiplications due to optimized integer kernels.
- **Lower Energy Consumption:** Reduced computation costs make quantization ideal for deployment on mobile and edge devices.

Trade-offs

- Accuracy may slightly degrade, especially in very low-bit quantization (e.g., INT4).
- Quantization schemes must be carefully chosen to avoid instability in sensitive layers such as attention and embeddings.

Code Example: PyTorch Dynamic Quantization

```
import torch
from transformers import AutoModelForSequenceClassification

# Load pre-trained model
```

```

model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased-finetuned-sst-2-english"
)

# Apply dynamic quantization (INT8 for Linear layers)
quantized_model = torch.quantization.quantize_dynamic(
    model, {torch.nn.Linear}, dtype=torch.qint8
)

print("Original size:", model.num_parameters() * 4 / 1e6, "MB")
print("Quantized size:", quantized_model.num_parameters() * 1 / 1e6, "MB")

```

Code Example: 4-bit Quantization with Bitsandbytes

```

from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

bnb_config = BitsAndBytesConfig(load_in_4bit=True,
                                bnb_4bit_use_double_quant=True,
                                bnb_4bit_quant_type="nf4")

model = AutoModelForCausalLM.from_pretrained(
    "TinyLlama/TinyLlama-1.1B-Chat-v1.0",
    quantization_config=bnb_config,
    device_map="auto"
)

```

Visual Illustration

Linear Quantization

An affine mapping of integers to real numbers $r = S(q - Z)$

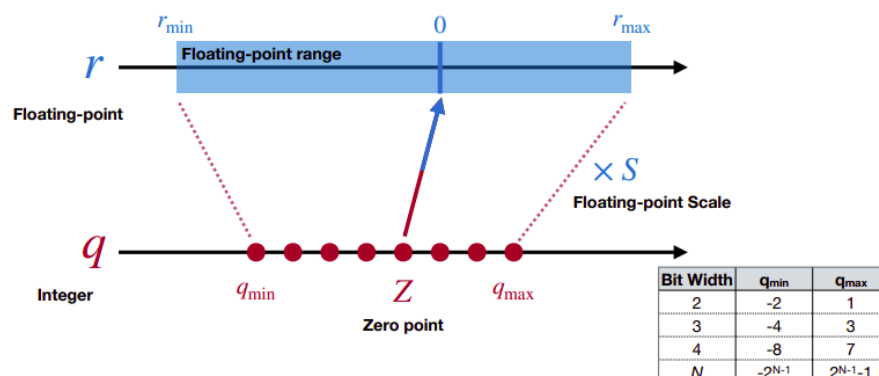


Figure 1: Illustration of quantization: mapping continuous floating-point values into discrete integer values.

Floating-Point Format Comparison

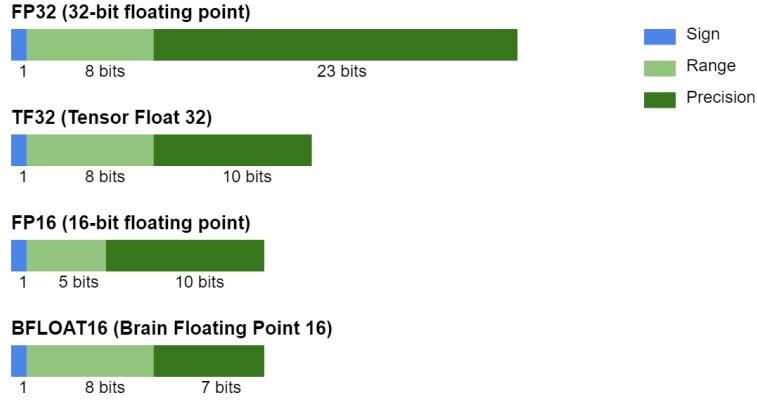


Figure 2: Accuracy vs Model Size trade-off for FP32, INT8, and INT4 quantization.

Conclusion

Quantization is a powerful technique to reduce the size and inference cost of large models like BERT and LLaMA. With methods such as post-training quantization, quantization-aware training, and weight-only quantization, models can be deployed efficiently on CPUs, GPUs, and edge devices. In practice, INT8 often provides an excellent balance between efficiency and accuracy, while INT4 enables even larger models to run within limited memory budgets.